

Plan de Integracion — Google Gemma 4 en AMS-Production

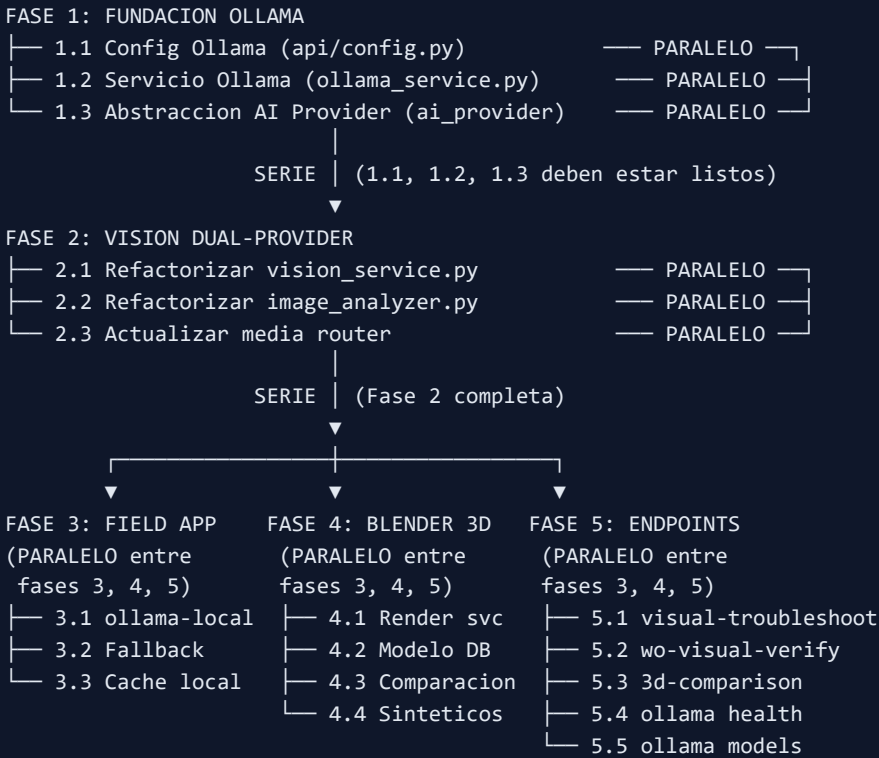
Documento vivo. El estatus de cada actividad se actualiza a medida que se completan. Fecha de inicio: 2026-04-06 | Branch: `Agentic_solutions_jose`

Convenciones

Estatus	Significado
PENDIENTE	No iniciado
EN PROGRESO	En desarrollo
COMPLETADO	Terminado y verificado
BLOQUEADO	Dependencia no resuelta

Ejecucion	Significado
PARALELO	Se puede ejecutar simultaneamente con otras tareas del mismo bloque
SERIE	Requiere que las dependencias esten completadas antes de iniciar

Mapa de Dependencias (Serie / Paralelo)



Casos de Uso

ID	Caso de Uso	Fases Requeridas	Modelo Recomendado
UC1	Troubleshooting visual offline	1, 2, 3, 5	E4B (8GB RAM)
UC2	Verificacion visual ejecucion OT	1, 2, 5	E4B / 26B
UC3	Inspeccion comparacion 3D vs foto	1, 2, 4, 5	26B (16GB RAM)
UC4	Digital Twin visual Blender	4	26B / 31B
UC5	AI offline-first plantas remotas	1, 2, 3	E4B (edge server)
UC6	Analisis audio nativo	1, 3	E2B / E4B

FASE 1 — FUNDACION OLLAMA EN BACKEND

Crear la capa de integracion con Ollama para usar Gemma 4 localmente.

1.1 Configuracion Ollama en Settings

Estatus	COMPLETADO
Ejecucion	PARALELO (con 1.2 y 1.3)
Archivo	api/config.py
Que hacer	Agregar al class Settings: <code>OLLAMA_BASE_URL</code> , <code>OLLAMA_MODEL</code> , <code>OLLAMA_TIMEOUT</code> , <code>AI_PROVIDER_DEFAULT</code> . Valores por defecto: <code>http://localhost:11434</code> , <code>gemma4</code> , <code>120</code> , <code>claude</code> .

1.2 Servicio Ollama Client

Estatus	COMPLETADO
Ejecucion	PARALELO (con 1.1 y 1.3)
Archivo	api/services/ollama_service.py (NUEVO)
Que hacer	Crear clase <code>OllamaClient</code> con httpx. Metodos: <code>health_check()</code> , <code>list_models()</code> , <code>generate()</code> , <code>chat()</code> , <code>analyze_image()</code> , <code>analyze_multiple_images()</code> . Singleton via <code>get_ollama_client()</code> . Soporte base64 para imagenes. JSON parsing con strip de markdown fences.

1.3 Abstraccion AI Provider

Estatus	COMPLETADO
Ejecucion	PARALELO (con 1.1 y 1.2)
Archivo	api/services/ai_provider_service.py (NUEVO)
Que hacer	Enum <code>AIPProvider</code> (CLAUDE, OLLAMA, AUTO). Funciones: <code>resolve_provider()</code> , <code>analyze_image()</code> , <code>generate_text()</code> . AUTO: intenta Claude, si no hay API key o internet, usa Ollama. Normaliza formatos request/response entre proveedores.

FASE 2 — VISION DUAL-PROVIDER

Refactorizar servicios de vision existentes para soportar Claude + Ollama.

2.1 Refactorizar vision_service.py

Estatus	COMPLETADO
Ejecucion	PARALELO (con 2.2 y 2.3)
Archivo	api/services/vision_service.py
Que hacer	Agregar parametro <code>provider: str = "auto"</code> a <code>analyze_images()</code> y <code>analyze_image()</code> . Delegar a <code>ai_provider_service.analyze_image()</code> en lugar de llamar directamente a anthropic. Mantener post- procesamiento de priority/activityClass.

2.2 Refactorizar image_analyzer.py

Estatus	COMPLETADO
Ejecucion	PARALELO (con 2.1 y 2.3)
Archivo	tools/processors/image_analyzer.py
Que hacer	Agregar clase <code>OllamaImageAnalysisService</code> con misma interfaz. Factory <pre>get_image_analysis_service(provider="auto")</pre> que retorna servicio Claude u Ollama segun disponibilidad. Reutilizar <code>_SYSTEM_PROMPT</code> y <code>_parse_response</code> (son provider-agnosticos).

2.3 Actualizar media router

Estatus	COMPLETADO
Ejecucion	PARALELO (con 2.1 y 2.2)
Archivo	api/routers/media.py
Que hacer	Agregar parametro <code>provider: str = Form(default="auto")</code> al endpoint <code>/media/analyze-image</code> . Pasar a <code>get_image_analysis_service(provider=provider)</code> .

FASE 3 — FIELD APP OFFLINE AI

Habilitar la PWA para usar Gemma 4 localmente cuando no hay internet.

3.1 Cliente Ollama para Browser

Estatus	COMPLETADO
Ejecucion	PARALELO (con 3.2 y 3.3)
Archivo	field_app/src/api/ollama-local-api.ts (NUEVO)
Que hacer	Crear cliente TypeScript que llama directamente a <code>http://localhost:11434/api/generate</code> . Funciones: <code>checkOllamaAvailable()</code> , <code>analyzeImageLocal(blob, context)</code> , <code>classifyTextLocal(text, tag)</code> . Manejo de CORS (Ollama permite all origins).

3.2 Logica de Fallback en Media API

Estatus	COMPLETADO
Ejecucion	PARALELO (con 3.1 y 3.3)
Archivo	field_app/src/api/media-api.ts
Que hacer	Agregar 3-tier fallback: 1) Ollama local, 2) Server API, 3) Queue offline. Detectar disponibilidad de Ollama antes de cada request. Cache resultado de health check por 30s.

3.3 Cache Local de Resultados AI

Estatus	COMPLETADO
Ejecucion	PARALELO (con 3.1 y 3.2)
Archivo	field_app/src/db/local-db.ts
Que hacer	Agregar tabla <code>aiResults</code> a Dexie schema: <code>localId</code> , <code>captureId</code> , <code>provider</code> , <code>resultType</code> , <code>resultJson</code> , <code>createdAt</code> , <code>syncedToServer</code> . Bump version a 4.

FASE 4 — BLENDER 3D PIPELINE

Conectar modelos 3D de equipos con el sistema de inspeccion visual.

4.1 Servicio de Renderizado Blender

Estatus	COMPLETADO
Ejecucion	PARALELO (con 4.2)
Archivo	api/services/blender_render_service.py (NUEVO)
Que hacer	Servicio que usa Blender MCP tools para renderizar equipos desde angulos estandar (frente, lado, arriba, isometrico). Funciones: <code>render_equipment(type, angle, resolution)</code> , <code>render_inspection_views(type)</code> . Almacena renders en <code>data/equipment_renders/</code> .

4.2 Modelo DB para Modelos 3D

Estatus	COMPLETADO
Ejecucion	PARALELO (con 4.1)
Archivo	api/database/models.py
Que hacer	Crear <code>Equipment3DModelModel</code> : <code>model_id</code> , <code>equipment_type_id</code> (FK), <code>blender_file_path</code> , <code>sketchfab_url</code> (nullable), <code>reference_renders</code> (JSON con paths por angulo), <code>created_at</code> . Migracion Alembic.

4.3 Servicio de Comparacion Visual

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 4.1 y 4.2)
Archivo	api/services/visual_comparison_service.py (NUEVO)
Que hacer	Cargar render de referencia (condicion ideal) + foto de campo (condicion real). Enviar ambas a Gemma 4 con prompt de comparacion. Retornar lista de desviaciones con severidad. Funciones: compare_reference_vs_field(equipment_type, field_photo_b64) .

4.4 Pipeline de Datos Sinteticos

Estatus	PENDIENTE
Ejecucion	SERIE (depende de 4.1)
Archivo	tools/processors/synthetic_image_generator.py (NUEVO)
Que hacer	Generar imagenes de entrenamiento con defectos simulados via Blender: corrosion (textura), desalineacion (transform), piezas faltantes (hide). Almacenar en data/synthetic_training/ .

FASE 5 — NUEVOS ENDPOINTS AGENTICOS

Exponer las nuevas capacidades como endpoints REST.

5.1 Endpoint Visual Troubleshooting

Estatus	COMPLETADO
Ejecucion	PARALELO (con 5.2, 5.3, 5.4, 5.5)
Archivo	api/routers/agenti c.py
Que hacer	POST /agentic/visual-troubleshooting . Request: image_base64, equipment_tag, plant_id, provider. Usa vision + equipment_library para diagnostico completo.

5.2 Endpoint WO Visual Verify

Estatus	COMPLETADO
Ejecucion	PARALELO (con 5.1, 5.3, 5.4, 5.5)
Archivo	api/routers/agenti c.py
Que hacer	POST /agentic/wo-visual-verify . Request: before_image_b64, after_image_b64, work_order_id. Compara antes/despues y verifica ejecucion correcta.

5.3 Endpoint 3D Comparison

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 4.3)
Archivo	api/routers/agenti c.py
Que hacer	POST /agentic/3d-comparison . Request: equipment_type, field_photo_b64. Carga render 3D de referencia y compara con foto real.

5.4 Endpoint Ollama Health

Estatus	COMPLETADO
Ejecucion	PARALELO (con 5.1, 5.2, 5.5)
Archivo	api/routers/agentic.py
Que hacer	GET /agentic/ollama/health . Retorna status de Ollama, modelos disponibles, modelo activo.

5.5 Endpoint Ollama Models

Estatus	COMPLETADO
Ejecucion	PARALELO (con 5.1, 5.2, 5.4)
Archivo	api/routers/agentic.py
Que hacer	GET /agentic/ollama/models . Lista modelos instalados en Ollama local con tamano y metadata.

INSTALACION DE GEMMA 4 (Pasos para el usuario)

Requisitos de Hardware

Modelo	RAM minima	GPU recomendada	Tamano descarga	Uso recomendado
E2B	5 GB	No necesaria	~2 GB	Moviles, tablets
E4B	8 GB	Cualquiera 4+ GB	~5 GB	Laptops campo
26B (MoE)	16-20 GB	12+ GB VRAM	~16 GB	Servidores edge
31B	20+ GB	24+ GB VRAM	~20 GB	Estaciones trabajo

Paso a Paso

```
# 1. Descargar e instalar Ollama desde https://ollama.com
#   Windows: ejecutar el .exe instalador

# 2. Verificar que Ollama esta corriendo
ollama --version

# 3. Descargar modelo recomendado (E4B, ~5 GB)
ollama pull gemma4

# 4. (Opcional) Modelo mas grande si hay hardware
ollama pull gemma4:26b

# 5. Probar que funciona
ollama run gemma4 "Describe los componentes principales de una bomba centrifuga"

# 6. Verificar API local
curl http://localhost:11434/api/tags

# 7. Instalar dependencia Python
pip install ollama
```

Capacidades de Gemma 4

- **Texto:** Razonamiento, codigo, JSON estructurado, function calling, 128K-256K contexto
- **Vision** (todos los modelos): OCR, deteccion objetos, analisis documentos, graficos
- **Audio** (solo E2B/E4B): Reconocimiento de voz, comprension de audio
- **Licencia:** Apache 2.0 — uso comercial sin restricciones

Resumen de Progreso

Fase	Total	Completados	Pendientes	%
Fase 1: Fundacion Ollama	3	3	0	100%
Fase 2: Vision Dual-Provider	3	3	0	100%
Fase 3: Field App Offline	3	3	0	100%
Fase 4: Blender 3D	4	3	1	75%
Fase 5: Endpoints	5	5	0	100%
TOTAL	18	17	1	94%

Pendiente: 4.4 Pipeline de Datos Sinteticos (requiere modelos 3D previos en Blender)

Casos de Uso Extendidos — Investigacion de Mercado

Basado en investigacion de implementaciones reales en mineria, oil & gas, y manufactura pesada.

CU-EXT-1: Deteccion de EPP por Vision (Safety Compliance)

Referencia	Visionify, viAct.ai
Descripcion	Gemma 4 analiza fotos/video de camaras existentes para detectar uso correcto de EPP: casco, chaleco, guantes, lentes, botas, proteccion auditiva. Alertas en tiempo real al supervisor.
Impacto	Reduccion de accidentes por incumplimiento de seguridad. Monitoreo 24/7 sin personal dedicado.
Hardware	Camaras IP existentes + Ollama en servidor edge
Integracion AMS	Vincular con <code>ExecutionChecklistModel</code> — gate de seguridad antes de iniciar OT

CU-EXT-2: Identificacion de Repuestos por Foto

Referencia	nyris (20,000+ clases), ITK Engineering, Partium
Descripcion	Tecnico fotografia pieza danada → Gemma 4 identifica tipo, dimensiones, material → busca codigo SAP en inventario. Elimina necesidad de descripcion manual.
Impacto	Reduccion de tiempo de busqueda de repuestos. Precision en codigos de material SAP.
Integracion AMS	Vincular con <code>InventoryItemModel</code> y catalogo de materiales de <code>vision_service.py</code> (10XXXXXX)

CU-EXT-3: OCR de Placas de Equipo y P&IDs

Referencia	Symphony AI, Azati AI, Microsoft ISE
Descripcion	Gemma 4 lee placas de identificacion de equipos (nameplate) con datos: fabricante, modelo, serial, potencia, RPM. Tambien puede digitalizar P&IDs en papel. Precision simbolos: 91.6%, caracteres: 83.1%.
Impacto	Digitalizacion rapida de activos. Verificacion de datos SAP vs placa real.
Integracion AMS	Auto-completar <code>HierarchyNodeModel</code> con datos de placa. Vincular con <code>equipment_library.json</code> .

CU-EXT-4: Deteccion de Fallas por Audio

Referencia	Sage Journal (pump fault detection), Nature Scientific Reports (bearing fault)
Descripcion	Gemma 4 E4B procesa audio de rodamientos, bombas, motores. Detecta patrones sonoros: golpeteo (bearing defect), cavitacion (pump), roce (seal), vibracion anomala. Sin sensores adicionales, solo microfono.
Impacto	Monitoreo de condicion no-invasivo. Complemento a analisis de vibracion CBM.
Integracion AMS	Nuevo canal de captura en <code>FieldCaptureModel</code> . Alimentar <code>FailurePredictionModel</code> .

CU-EXT-5: Verificacion LOTO por Vision

Referencia	AWS Architecture Blog (safety monitoring)
Descripcion	Antes de iniciar OT, tecnico fotografia el equipo. Gemma 4 verifica: candados LOTO visibles, tarjetas de bloqueo, equipo des-energizado (indicadores apagados), area despejada.
Impacto	Prevencion de accidentes por falta de LOTO. Evidencia digital de cumplimiento.
Integracion AMS	Gate en <code>ExecutionChecklistModel.is_gate=True</code> . Bloquear avance sin verificacion visual.

CU-EXT-6: Inspeccion de Corrosion con Drones + AI

Referencia	Oil & gas pipeline inspection (80% ganancia eficiencia, 50-70% reduccion costo)
Descripcion	Imagenes de drone de estructuras, tanques, tuberias procesadas por Gemma 4 para detectar corrosion, deformacion, grietas. Sin acceso en altura.
Impacto	Inspeccion de areas inaccesibles. Reduccion de riesgo para tecnicos.
Integracion AMS	Alimentar <code>RBIAssessmentModel</code> (Risk Based Inspection). Generar WR automaticos.

CU-EXT-7: Digital Twin Interactivo con Blender

Referencia	Siemens Industrial AR, PTC Step Check
Descripcion	Modelo 3D Blender del equipo con estado de salud superpuesto. Colores por severidad en cada componente. Click en sub-ensamble muestra historial de fallas, ultima inspeccion, proxima tarea.
Impacto	Visualizacion espacial del estado de mantenimiento. Decision-making visual para planificadores.
Integracion AMS	Vincular <code>HierarchyNodeModel</code> + <code>HealthScoreModel</code> + <code>Equipment3DModelModel</code> .

CU-EXT-8: Entrenamiento de Tecnicos con Escenarios 3D

Referencia	JigSpace, Roundtable Learning (15% reduccion errores)
Descripcion	Renderizar secuencias de desarme/arme de equipos en Blender. Gemma 4 genera instrucciones paso-a-paso desde renders. Tecnico practica antes de ir al campo.
Impacto	Reduccion de errores en ejecucion. Tecnicos nuevos productivos mas rapido.
Integracion AMS	Vincular con <code>WorkInstructionModel</code> (work packages M3). Generar instrucciones visuales.

CU-EXT-9: Edge AI en Plantas Remotas

Referencia	NVIDIA Jetson (Orin Nano \$200, AGX Xavier Industrial), Barbara Platform
Descripcion	Mini-PC ruggedizado (Jetson Orin) con Ollama + Gemma 4 E4B en planta minera remota. Procesamiento local con 87.2% menos latencia que cloud. Sub-100ms respuesta.
Hardware	NVIDIA Jetson Orin Nano (~\$200) o Intel NUC industrial
Impacto	AI disponible 24/7 sin internet. Reduccion 20% downtime no planificado (Deloitte).
Integracion AMS	FastAPI + Ollama en mismo dispositivo. Sync batch con servidor central.

Matriz de Priorizacion de Casos de Uso

Caso de Uso	Complejidad	Impacto	Hardware Adicional	Priorizacion
UC1: Troubleshooting visual offline	Baja	Alto	Ninguno (laptop)	INMEDIATO
UC2: Verificacion visual OT	Baja	Alto	Ninguno	INMEDIATO
CU-EXT-2: Identificacion repuestos	Baja	Alto	Ninguno (celular)	INMEDIATO
CU-EXT-3: OCR placas equipo	Baja	Medio	Ninguno (celular)	INMEDIATO
CU-EXT-5: Verificacion LOTO	Media	Alto	Ninguno	CORTO PLAZO
CU-EXT-1: Deteccion EPP	Media	Alto	Camaras IP existentes	CORTO PLAZO
CU-EXT-4: Deteccion audio	Media	Alto	Microfono (\$20)	CORTO PLAZO
UC3: Comparacion 3D vs foto	Media	Medio	Modelos Blender	MEDIANO PLAZO
CU-EXT-9: Edge AI plantas remotas	Alta	Muy Alto	Jetson Orin (\$200)	MEDIANO PLAZO
CU-EXT-6: Inspeccion drones	Alta	Alto	Drone + camara	MEDIANO PLAZO
UC4: Digital Twin Blender	Alta	Medio	Modelos Blender	LARGO PLAZO
CU-EXT-7: Digital Twin interactivo	Alta	Alto	Modelos Blender	LARGO PLAZO
CU-EXT-8: Entrenamiento 3D	Alta	Medio	Modelos Blender	LARGO PLAZO